

ANTIPATRONES

Definición: Un antipatrón es una "solución" a un problema que genera consecuencias negativas inesperadas, generalmente manifestadas en etapas posteriores del desarrollo.

Características:

- Parece una solución viable inicialmente
- Genera problemas a largo plazo
- Requiere dos soluciones: la inicial y la correctiva (refactoring)

Spaghetti Code

Definición: Código complejo, desorganizado y no documentado que es difícil de mantener.

Causas: Desconocimiento, flojera

Síntomas:

- Métodos sin parámetros
- Variables globales innecesarias
- Nombres de objetos que son nombres de procesos
- Mínima relación entre objetos
- Dificultad para reutilizar código
- No uso de herencia o polimorfismo

Spaghetti Code - todo en una función

```
def procesar_todo():
```

```
    global datos
```

```
    # 200 líneas de código mezclando lógica, UI y datos
```

```
    # sin estructura clara
```

Puntos en contra:

- Mantenimiento extremadamente difícil
- Imposible de escalar
- Alto costo de modificaciones

The Blob (Clase Dios)

Definición: Una clase que acumula demasiadas responsabilidades que deberían distribuirse en múltiples clases.

Causas: Prisa, flojera

Síntomas:

- Clase con muchos atributos y métodos (>60)
- Colección de atributos no relacionados
- Carece de diseño orientado a objetos
- Relación mínima entre objetos

// The Blob - una clase que hace todo

```
public class SistemaCompleto {  
  
    private Usuario usuario;  
  
    private Producto producto;  
  
    private Factura factura;  
  
    private Configuracion config;  
  
  
    public void login() { /* ... */ }  
  
    public void generarFactura() { /* ... */ }  
  
    public void enviarEmail() { /* ... */ }  
  
    public void generarReporte() { /* ... */ }  
  
    // ... 50 métodos más  
}
```

Puntos en contra:

- Viola el principio de responsabilidad única
- Dificulta las pruebas unitarias
- Acoplamiento excesivo

Cut & Paste

Definición: "Reutilización" de código mediante copiar y pegar fragmentos de código.

Causas: Flojera

Síntomas:

- Mismo error aparece en múltiples secciones
- Alto costo de mantenimiento

Ejemplo:

```
// Misma validación copiada en 10 archivos diferentes

function validarEmail(email) {

    // 15 líneas de validación

    // Si hay un error, hay que corregirlo en 10 lugares
}
```

Puntos en contra:

- Duplicación de código
- Corrección de errores multiplicada
- Inconsistencias en el sistema

Golden Hammer

Definición: Usar una única herramienta/tecnología para todos los problemas.

Causas: Ignorancia, orgullo, mente cerrada

Síntomas:

- Problemas de desempeño y escalabilidad
- Arquitectura descrita por un único producto
- Requerimientos acomodados al producto
- Conocimiento muy específico

Ejemplo:

- Usar Excel para todo (base de datos, interfaz, reporting)
- Usar siempre la misma base de datos sin evaluar alternativas
- Aplicar el mismo patrón de diseño en todos los casos

Puntos en contra:

- Soluciones inadecuadas para problemas específicos
- Falta de innovación
- Dependencia excesiva de una tecnología

Lava Flow

Definición: Código legado sin documentación o capacidad de mantenimiento.

Causas: Avaricia, envidia, flojera

Síntomas:

- Falta de documentación
- Proyectos experimentales en producción
- Variables y código sin justificación
- Código comentado sin razón

Ejemplo:

```
python
```

```
# Código de hace 5 años que nadie entiende
```

```
def calcular_algo(x):
```

```
    # factor_magico = 42.7 ¿por qué?
```

```
    factor_magico = 42.7
```

```
    resultado = x * factor_magico + 13
```

```
    # no tocar - sistema se cae si se modifica
```

```
    return resultado
```

Puntos en contra:

- Miedo a modificar el código
- Dependencia de personas específicas
- Alto riesgo de cambios

Obsolescencia Continua

Definición: Dificultad para actualizar librerías y tecnologías debido a cambios rápidos.

Causas: Entorno cambiante

Síntomas:

- Poco uso de estándares abiertos
- Aplicación no actualizada por mucho tiempo

Ejemplo:

- Sistema usando jQuery 1.4 en 2024
- Framework descontinuado hace 5 años
- Dependencias con vulnerabilidades conocidas

Puntos en contra:

- Vulnerabilidades de seguridad
- Compatibilidad issues
- Falta de soporte

3. Conclusión

Los antipatrones representan soluciones aparentemente rápidas que generan problemas a largo plazo. Su identificación temprana y refactoring son esenciales para mantener sistemas saludables y mantenibles.

Recomendaciones:

- Realizar revisiones de código periódicas
- Documentar las decisiones de diseño
- Mantener las dependencias actualizadas
- Promover buenas prácticas de programación
- Invertir en capacitación del equipo